



UNIVERSIDAD
DE BURGOS

ESCUELA POLITÉCNICA SUPERIOR

INFORME TÉCNICO

PRÁCTICA 3: PROGRAMACIÓN CON SOCKETS

Cliente en C++ y Servidor en Python



ASIGNATURA

Sistemas Operativos



AUTORES

Ander Zuazquita Pastor
Lucia Castellanos Paz



FECHA

8 de mayo de 2026



INDICE

1. Introduccion y contextualización del problema
2. Explicaion del funcionamiento del programa desarrollado
3. Explicacion del uso de arrayas y matrices
4. Explicacion de la lectura del archivo de entrada
5. Explicacion de la escritura del archivo de salida
6. Explicaion del script en Bash para la ejecución mediante sh
7. Capturas de pantalla de la ejecución del programa
8. Interpretacion de los resultados obtenidos
9. Conclusiones
10. Bibliografia

1. INTRODUCCION Y CONTEXTUALIZACION DEL PROBLEMA

En esta practica hemos desarrollado un sistema de comunicación cliente-servidor utilizando sockets TCP/IP.

El objetivo de la practica es implementar un cliente en C++ capaz de conectarse a un servidor desarrollado en Python para recibir datos simulados de una cámara térmica industrial.

En esta practica se ha usado el protocolo TCP/IP, el cual garantiza una transmisión fiable de información.

Se han usado las siguientes herramientas para realizar esta practica:

- Sockets TCP/IP
- Programación en C++
- Programación en python
- Uso de Bash
- Gestión de archivos
- Uso de GitHub

El sistema simula un escenario donde un dispositivo remoto transmite información térmica a un cliente encargado de almacenar en un archivo local.

2. EXPLICAION DEL FUNCIONAMIENTO DEL PROGRAMA DESARROLLADO

El funcionamiento del proyecto se basa en una arquitectura cliente-servidor.

El servidor de Python actúa como una cámara térmica simulada que genera datos numéricos y los envía mediante sockets TCP/IP.

Mientras, el cliente en C++ realiza lo siguiente:

- Crear un socket TCP/IP
- Configura la dirección IP
- Establece la conexión con el servidor
- Recibir los datos enviados mediante la función `recv()`
- Almacenar los datos en memoria
- Guardar los datos en un archivo de texto
- Mostrar los resultados en pantalla

El flujo del funcionamiento es el siguiente:

Servidor Python → sockets TCP/IP → Cliente C++ → Archivo local

El servidor genera una serie de datos simulados y los envía al cliente usando una conexión TCP/IP.

El cliente recibe los datos y los almacena en un vector dinámico para guardarlos en un archivo llamado `datos_recibidos.txt`.

Además el programa usa funciones independientes para mejorar la organización:

- `crearSocket()`
- `configurarServidor()`
- `conectarServidor()`
- `recibirDatos()`
- `guardarDatos()`
- `imprimirDatos()`

3. EXPLICAION DEL USO DE ARRAYS Y VECTORES

Al realizar la practica usamos estructuras dinámicas para almacenar los datos recibidos desde el servidor.

Vector<unsigned char> datos;

El uso de vectores nos da las siguientes ventajas:

- Permite almacenar datos
- No necesita saber el tamaño exacto con antelación
- Facilita la entrada de elementos
- Simplifica la gestión de memoria

Cada dato recibido mediante `recv()` se añade al vector usando `push_back()`.

```
while (recv(sockfd, &buffer, sizeof(buffer), 0) > 0) {  
    datos.push_back(buffer);  
}
```

En el servidor Python también usa un array para generar los datos simulados.

```
data_array = [i % 256 for i in range(768)]
```

Este array contiene valores que después son enviados al cliente.

4. EXPLICACION DE LA LECTURA DEL ARCHIVO DE ENTRADA

En la practica no se usa un archivo de entrada normal.

La lectura de datos se realiza desde el socket TPC/IP mediante la función `recv()`

La función `recv()` nos permite recibir información enviada desde el servidor.

`recv(sockfd, &buffer, sizeof(buffer), 0)`

El cliente permanece recibiendo datos mientras el servidor sigue enviando información.

Cada dato recibido se almacena en la variable `buffer` y se introduce en el vector dinámico.

Este mecanismo permite realizar una comunicación en tiempo real entre el servidor y el cliente.

5. EXPLICACION DE LA ESCRITURA DEL ARCHIVO DE SALIDA

Una vez recibidos los datos el cliente los guarda en un archivo usando una librería ofstream.

El archivo generado se llama:

output/datos_recibidos.txt

La escritura del archivo se realiza usando la función guardarDatos().

```
ofstream archivo("output/datos_recibidos.txt");
```

Despues, cada valor almacenado se escribe línea por línea:

```
for (unsigned char dato : datos) {  
    archivo << (int)dato << endl;  
}
```

Finalmente el archivo se cierra usando:

```
archivo.close();
```

Con este proceso, todos los datos enviados quedan almacenados localmente.

6. EXPLICACION DEL SCRIPT BASH PARA LA EJECUCION MEDIANTE SH

Para automatizar el proceso de compilación del cliente, se creo un script Bash llamado run.sh.

El objetivo del script consiste en simplificar la ejecución del programa.

El script realiza lo siguiente:

- Compilar el cliente C++
- Comprobar que la compilación fue correcta
- Ejecuta el programa cliente
- Muestra mensajes de control

Para poder ejecutar el script se debe otorgar permisos de ejecución:

chmod +x run.sh

Y así el script puede ejecutarse mediante:

./run.sh

7. CAPTURAS DE PANTALLA DE LA EJECUCION DEL PROGRAMA

SERVIDOR PYTHON EJECUTANDOSE

```
python3 ServidorCamara.py
anderzp in @anderHP in :~/Escritorio $ cd practica3_so
anderzp in @anderHP in :~/Escritorio/practica3_so $ cd server
anderzp in @anderHP in :Escritorio/practica3_so/server $ python3 ServidorCamara.py
Servidor escuchando en 127.0.0.1:12345
```

CLIENTE C++ EJECUTANDOSE

```
~/Escritorio/practica3_so
anderzp in @anderHP in :~/Escritorio $ cd practica3_so
anderzp in @anderHP in :~/Escritorio/practica3_so $ ./run.sh
compilando cliente...
compilacion correcta
ejecutando cliente..
Conexion establecida correctamente
Datos guardados correctamente

Datos recibidos:
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
~/Escritorio/practica3_so ~/Escritorio/practica3_so/serv.. ~/Escritorio/practica3_so
```

ARCHIVO GENERADO

```

251
252
253
254
255
anderzp in @anderHP in :~/Escritorio/practica3_so $ cat output/datos_recibidos.txt
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
~/Escritorio/practica3_so ~/Escritorio/practica3_so/serv... ~/Escritorio/practica3_so

```

REPOSITORIO GITHUB

output	implementacion sockets	2 days ago
screenshots	editar README y añadir capturas	8 minutes ago
server	implementacion cliente	2 days ago
src	eliminar archivo duplicado	3 minutes ago
.gitignore	Initial commit	2 days ago
README.md	editar README y añadir capturas	8 minutes ago
cliente	arreglo de cliente y sockets	2 days ago
run.sh	implementacion sockets	2 days ago

8. INTERPRETACION DE LOS RESULTADOS OBTENIDOS

Los resultados obtenidos fueron los esperados.

El cliente consiguió conectarse correctamente al servidor mediante sockets TCP/IP.

La comunicación entre ambos programas funciona correctamente y los datos enviados desde el servidor se reciben sin errores.

- La conexión TCP/IP es estable
- Recv() funcionaba correctamente
- Los datos se almacenan en memoria
- El archivo de salida se genera correctamente
- El script bash automatiza todo sin errores

También se comprobó el funcionamiento del repositorio GitHub usado para el trabajo.

El sistema reproduce de forma más sencilla el comportamiento de un entorno industrial basado en monitorización remota.

9. CONCLUSIONES

Durante el desarrollo del proyecto tuvimos diferentes problemas técnicos que pudimos resolver progresivamente.

El primer problema que tuvimos fue que por algún error del ordenador o por haber tocado algo en algún momento, el entorno WSL no funcionaba por la virtualización, pero se arreglo fácilmente.

Tambien hubo dificultades para acceder correctamente al proyecto desde diferentes terminales porque algunas se abrían en PowerShell en vez de Ubuntu, pero lo solucionamos con el comando **wsl** y accediendo al directorio correcto.

Otro de los problemas fue la compilación cliente C++, porque en un primer intento aparecion el error “undefined reference to main” debido a que el archivo cliente.cpp no contenía correctamente la función main() o el archivo no había sido guardado correctamente. Lo solucionamos porque sin querer creamos dos aaarchivos llamados Cliente.cpp y cliente.cpp y llamábamos al vacio que era Cliente.cpp en vez de cliente.cpp.

Durante las pruebas tambien vimos que era necesario ejecutar de forma simultanea el servidor Python y el cliente C++ en diferentes terminales para permitir la comunicación TCP/IP. Una vez lo entendimos el programa funciono correctamente.

Ademas hay un punto que no comprendemos y que en un principio no funcionaba, y es que la dirección IP que salía en el archivo no funcionaba, y si poníamos la de nuestro ordenador, si cambiábamos de dispositivo dejaba de funcionar, a si que pusimos una dirección proporcionada por chatgpt que no comprendemos aun porque hace que funcione.

10. BIBLIOGRAFIA

Explicacion del funcionamiento de sockets TCP/IP:

https://ocw.unican.es/pluginfile.php/2343/course/section/2300/PIR-Practica4_ManualSocketsC.pdf

Funcionamiento de la función rev():

<https://pubs.opengroup.org/onlinepubs/007904975/functions/recv.html>

<https://learn.microsoft.com/es-es/windows/win32/api/winsock/nf-winsock-recv>

Offstream en C++:

<https://cplusplus.com/doc/tutorial/files>